

● Expenser Full-Stack Application

Technical Architecture & Authentication

A deep dive into the technology stack, application pipeline, and
secure login architecture.

Project: Expenser

Focus: Backend, Assets, Auth

Format: Presenter Deck

SPEAKER NOTES

📄 SLIDE 1: TITLE & PRESENTATION INTRODUCTION

"Hello everyone, and welcome. Today we are doing a technical deep dive into **Expenser**, which is a premium full-stack group expense-sharing and debt-simplification application.

In this presentation, I am going to walk you through our selected technology stack—explaining why each component was chosen for speed and local independence—and then focus specifically on our custom dark-themed user login flow, looking at the Breeze authentication system, routing middlewares, and session security safeguards."

▶ **START PRESENTATION**

💡 **Highlight: Local Independence & Security**

Our Curated Core Stack

We chose a lightweight, modern, and highly modular technology stack optimized for local execution speed and relational data integrity.

PHP

Laravel 12 Engine

Robust MVC structure, Eloquent ORM, middleware, Breeze Auth.

SQL

SQLite Database

High-speed, zero-config relational local file database.

JS

Vite Asset Pipeline

Fast hot-reload compilation of customized CSS and JS bundles.

CSS

Vanilla CSS & Alpine

Clean, custom HSL styling and micro-interactions without bloat.

Core Stack

Slide 2

SPEAKER NOTES

SLIDE 2: TECHNOLOGY STACK OVERVIEW

"Let's look at the foundational components of Expenser. We selected a highly optimized, standard stack:

1. **Laravel 12** serves as our backend core, providing secure routing, controller abstractions, session control, and the Eloquent ORM.
2. **SQLite** provides a fast, zero-configuration local database that allows the app to run independently of heavy external servers while maintaining strict foreign key constraints.
3. **Vite** is our build system, handling hot module replacement during development and compiling highly compressed assets for production.
4. **Vanilla CSS & AlpineJS** let us create a custom dark-themed UI without importing heavy design libraries. This keeps page sizes incredibly small."

👉 Explain why SQLite is excellent for local dev

💡 Highlight: Performance & Custom CSS

Data Schema & Relational Integrity

Expenser structures data across 5 distinct tables using strict SQLite foreign keys and cascade triggers to prevent database orphans.

- **Groups & User Pivot:** A many-to-many relationship (`group_user`` table) tracks group members and joining dates.
- **Expenses:** Belongs to a group and links to a specific paying user (`paid_by``).
- **Expense Shares:** Stores individual user cost-shares, supporting equal, exact, or percentage splits.
- **Settlements:** Records cash transfers between a debtor (`paid_by``) and creditor (`paid_to``).

DATABASE SCHEMAS:

users: id, name, email, password_hash

groups: id, name, description, created_by

group_user: group_id, user_id, joined_at

expenses: id, group_id, paid_by, amount, split_type

expense_shares: id, expense_id, user_id, share_amount

settlements: id, group_id, paid_by, paid_to, amount

SPEAKER NOTES

SLIDE 3: DATABASE & RELATIONSHIPS

"Now let's review our relational model. A primary challenge in expense splitting is handling complex, flexible shares.

To resolve this, we NORMALIZED our schema into five tables. Users and Groups share a many-to-many relationship managed through a pivot table. Every expense is owned by a group and paid by one user. Crucially, the individual split records are moved to an `expense_shares`` table. This isolation is what allows Expenser to calculate arbitrary split amounts—whether equal, exact, or percentage-based—without polluting the main expenses table.

Settlements exist as a separate ledger, allowing users to clear balances through peer-to-peer record tracking."

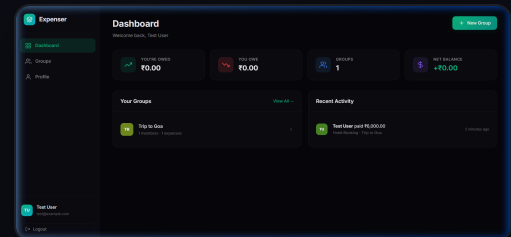
💡 Explain the `expense_shares` table isolation

👉 Slide transition: Moving to client asset delivery

Bespoke Design & Vite Compilation

Expenser bypasses generic component frameworks, delivering customized HSL CSS rules directly through Vite's production bundler.

- **Dark UI System:** Uses variables for background scales, glass opacity borders, and neon-emerald accents.
- **Vite Bundler:** Compiles CSS and Javascript source files, outputting production-ready files in `public/build/assets/`.
- **Server Delivery:** When compiling for production, removing the temporary `public/hot` file tells Laravel to serve assets statically from the build folder.



SPEAKER NOTES

SLIDE 4: CSS DESIGN & VITE COMPILATION

"Our styling strategy is custom-designed. We did not use standard Tailwind templates or Bootstrap. Instead, we built a bespoke dark CSS framework from scratch using HSL color tokens. This gives us total control over details like borders, glowing elements, and responsive flexboxes.

Vite is used to compile and bundle our assets. When Vite compiles for production, it places the assets in the `public/build` directory and creates a `manifest.json`. During development, Vite creates a `public/hot` file to run a hot-reloading dev server. If that server is stopped, the presence of the stale `hot` file causes assets to fail. Removing the `hot` file instructs Laravel's `@vite` helper to serve the compiled, static CSS and JS files instead."

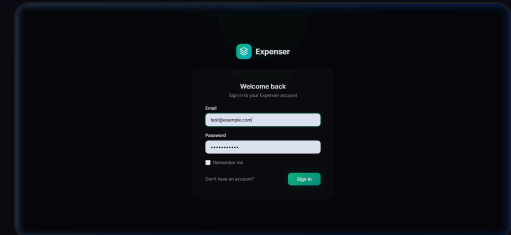
👉 Explain: Stale public/hot vs Static assets

💡 Highlight: Custom HSL gradients & Glassmorphism

Breeze Authentication Lifecycle

Expenser's secure entry-point relies on Laravel Breeze's session authentication backend, wrapped in our custom dark layout.

- **Credential Entry:** Secure login form submits the user's email and password to the server.
- **Database Match:** Laravel hashes the input and matches it against the stored bcrypt hash in SQLite.
- **Session Cookie:** If valid, Laravel starts a secure database session and sends an encrypted cookie to track the user.
- **Middleware Route Protection:** Non-authenticated users trying to access dashboard routes are redirected back to the login page.



SPEAKER NOTES

SLIDE 5: LOGIN FLOW & BREEZE SETUP

"Let's move on to the authentication flow. We implemented Laravel Breeze to handle user login, registration, and password recovery.

We customized Breeze's views with our dark theme. When a user logs in, their credentials are submitted over POST to the `login` route. Laravel hashes the password and verifies it against the hashed record in SQLite. Once validated, it starts a secure session (configured in `.env` to store sessions in the database) and sets a session cookie in the user's browser. This session cookie is verified on every subsequent request."

[👉 Show how guest.blade.php handles login views](#)[💡 Highlight: Clean Session-Based Security](#)

Active Security & Rate Limiting

Expenser builds security directly into the HTTP pipeline to protect user accounts and prevent brute-force attacks.

- **CSRF Injection:** Every login form generates a unique session token to block Cross-Site Request Forgery.
- **Attempt Throttling:** Built-in rate limiter restricts users to 5 login attempts per minute, locking out IPs on failure.
- **Password Salting:** Passwords are cryptographically salted and hashed using bcrypt (12 cost factor rounds).
- **Auth Guard Middleware:** The `auth` middleware protects all routes under `/dashboard` and `/groups`.

```
// Routes group with auth
middleware
Route::middleware(['auth',
'verified'])->group(function ()
{
    // Authenticated routes
    Route::get('/dashboard',
[DashboardController::class,
'index'])->name('dashboard');
    Route::resource('groups',
GroupController::class);
});
```

SPEAKER NOTES

SLIDE 6: LOGIN SECURITY & MIDDLEWARE DETAILS

"Security is integrated directly into our routing pipeline.

First, **CSRF protection** is enforced on all post requests. If a request is submitted without a valid token, Laravel immediately returns a 419 error.

Second, we use **Login Rate Limiting** to prevent brute-force attacks. The system allows a maximum of five login attempts per minute. If exceeded, it locks the account-IP combination for 60 seconds.

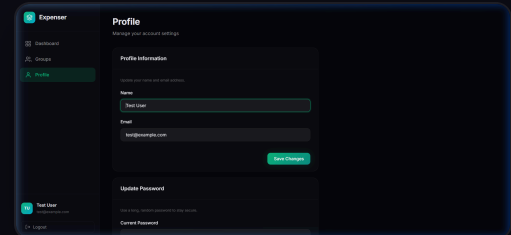
Third, passwords are hashed using **Bcrypt** with a work factor of 12.

Finally, we use the `auth` middleware to restrict access to sensitive routes, ensuring that unauthenticated users cannot view group expenses or settle debts."

Lifecycle: Registration to Logout

The application ensures a seamless, secure flow as users register, authenticate, view records, and log out.

- **Registration:** New users create an account; passwords are hashed and saved.
- **Authentication:** Registered users log in; a session is initialized and session cookies are sent.
- **Secure Session:** Users browse groups, log expenses, and settle balances under middleware protection.
- **Logout:** Destroys the server session, invalidates cookies, and redirects the user back to the landing page.



User Lifecycle

Slide 7

SPEAKER NOTES

SLIDE 7: AUTHENTICATION LIFECYCLE SUMMARY

"To conclude our authentication deep dive, let's trace the full lifecycle. A new user registers, hashes their credentials, and is logged in.

While logged in, they can update their profile information or change their password securely. When they log out, the session is deleted from the database and the session cookies are cleared, redirecting the user back to the landing page.

This ensures a complete, secure lifecycle from registration to logout."

[FINISH PRESENTATION](#)[💡 Highlight: Clean authentication lifecycle](#)